

Caregiver Signup Hardening — Implementation Plan

Evia / Codex handoff

July 7, 2026

CONTENTS

Caregiver Signup Hardening — Implementation Plan (for Codex)	
0. Context an implementer must know first	
Current flow (for orientation)	
A. Create the webapp account at phone entry (web flow) — PRIMARY ASK	
B. Stop texting dead placeholder links on API failure — HIGH	
C. Unblock non-clear background checks + fix swallowed clear notice — MED	
D. Text a webapp login link so caregivers reach their dashboard — MED	
E. Short bio silently wiped — LOW (cheap)	
F. MVR mis-config leaves caregiver uninformed — LOW (cheap)	
Deferred (documented, NOT in this plan — founder decision required to include)	
Suggested implementation order	
Definition of done	

Caregiver Signup Hardening — Implementation Plan (for Codex)

Audience: an implementing agent (Codex CLI) with full repo access. **Repo root:**

CareConnecxx-main/ (functions workspace: CareConnecxx-main/functions/). **Goal:** make the caregiver signup flow complete one-shot with no dead links, no silent stalls, the webapp account created the moment the user verifies their phone, and every required link delivered.

Do **not** deploy. Implement, build, run tests, and report. Deploy is the founder's call.

0. Context an implementer must know first

- **Product:** “Evia” (formerly Cara/CareConnex). React + TS SPA + Firebase Cloud Functions. Caregivers onboard conversationally over SMS via the Linq webhook. See root `CLAUDE.md`.
- **Caregiver onboarding is SMS-driven** after a web phone-verify handoff. Sole path: `functions/src/agents/onboardingConversation.ts` (`caregiver_*` steps).
- **LIVE PATH CAVEAT (verified in `functions/.env`):**

```
ONBOARDING_AGENT_LOOP=client,caregiver ,
ONBOARDING_AGENT_LOOP_COHORT_PCT=100
```

Caregivers run the **agent-native loop** for the *conversational collection* portion (name → bio). The loop falls back to the scripted `handleOnboardingStep` switch on any failure before a reply is sent.

 - **Side-effect steps are scripted on BOTH paths:** photo, documents, membership (Stripe), background check (Checkr), Stripe Connect, permissions, account creation, and all webhook-driven advancement (`advanceOnboardingStep`) run through the scripted handlers regardless of the agent loop. → **Fixes A, B, C, D below are on the live path.**
 - **Collection steps (bio, MVR ask) may be agent-loop-handled**, not the scripted `handleCaregiverAskBio` /MVR handler. → **For fixes E and F, patch the scripted handler AND verify/patch the agent-loop equivalent (`caregiverFieldAbsorber.ts` / `onboardingContract.ts`), or the fix only helps the fallback path.**

- **Evia AI rules (MANDATORY, from CLAUDE.md):** never parse free-form SMS intent with `regex/ .includes() /keyword` arrays — use `parseWithClaude()` / `quickComplete()`. Strict `YES / NO` binary is allowed only when the prompt explicitly said “Reply YES or NO”. Every new user-facing line should read like Evia (warm, short); prefer `generateCaraMessage(...)` for non-deterministic copy, matching the surrounding code.
- **Caregiver name field:** stored as `onboardingData.name` and `caregivers.name` (NOT `firstName`; `firstName` is client-only). Do not introduce `firstName` on caregiver docs.
- **Send primitive:** `sendMessage(chatId, text | {parts:[...]}, opts?)` in `functions/src/linq/client.ts`. A plain string with a URL auto-splits into a text bubble + a rich link bubble. A hard transport failure is dead-lettered to `linq_outbound_queue` for redelivery. Link bubbles use `{ parts: [{ type: "link", value: url }] }`.
- **App URL helpers:** `functions/src/config/appUrl.ts` — `getAppUrl()` (default `https://www.eviacares.com`) and `appLink(path)`.
- **Admin paging:** write to `admin_alerts` collection (`{ type, severity, ...ctx, createdAt: ISO, resolved: false }`) — follow existing call shapes in `onboardingConversation.ts`.
- **Build/test:**

```
npm --prefix functions run build
npm --prefix functions test -- --run # or: npm test -- --run at repo root
for SPA
```

Existing known-failing tests: `onboardingReplay` has 3 pre-existing failures — do not attribute to this work.

CURRENT FLOW (FOR ORIENTATION)

```

/start?role=caregiver → Firebase Phone Auth OTP → createWebOnboardingSession
(index.ts:287)
  writes web_onboarding_sessions ONLY — NO account doc yet
    ↓ user texts "Hey Evia"
Linq webhook → agent_sessions/{phone} → caregiver steps:
  name → location(SCC-gated) → story → experience → specialties → profile →
  availability → job_type → rate → email → bio
  → PHOTO (ensureWebAccount: Auth user + users/{uid} created HERE, ~12 turns
in)
  → documents → MVR? → MEMBERSHIP($66.49 Stripe) → BG-CHECK(Checkr)
    (caregivers/{uid} pre-created here)
  → STRIPE CONNECT payout (caregivers/{uid} finalized via webhook)
  → permissions (2 Q) → sends /p/{id} profile link

```

A. Create the webapp account at phone entry (web flow) — PRIMARY ASK

Problem: the `users/{uid}` account doc is not created until the photo step (`ensureWebAccount`, ~12 SMS turns after phone entry). Founder wants the account to exist the instant the user enters their phone number to get access to Evia.

Decisions already made by founder:

- Web (`/start`) flow only. Cold-SMS (“Hey Evia” with no website visit) is **out of scope** — leave its current behavior.
- Create `users/{uid}` only. Do **NOT** create `caregivers/{uid}` early (it’s entangled with the Checkr `candidate_id` and Stripe Connect finalization — larger blast radius).

Why it’s low-risk: at phone-verify the Firebase Auth user already exists (client SDK `confirm()`), and `createWebOnboardingSession` already holds `context.auth.uid`, `phone`, `role`, and sanitized `name`. Only the Firestore `users/{uid}` doc is missing.

Change — `functions/src/index.ts`, inside `createWebOnboardingSession` (starts line 287): After the existing `web_onboarding_sessions/{phone}` write (line ~335-345) and before the referral block (or after it — no dependency), add an idempotent merge write

to `users/{uid}`:

- Doc id = `context.auth.uid`.
- Fields: `uid`, `phone`, `updatedAt: serverTimestamp()`.
- Seed `userType: role` **only if the doc has no existing `userType`** (never flip an admin/existing account — mirror the guard in `ensureWebAccount` at `onboardingConversation.ts:275`).
- Seed the name **only if present**: `role === "caregiver" ? { name } : { firstName: name }`.
- Seed `createdAt: serverTimestamp()` only when the doc does not already exist (read-before-write, as `ensureWebAccount` does).
- Wrap in try/catch; on failure write an `admin_alerts` entry `type:"auth_account_create_failed"` (reuse the exact shape from `ensureWebAccount` at `onboardingConversation.ts:286-294`) and **still return success** — the bridge doc write is what the flow depends on; account seeding must not block signup.
- **Do not** call `admin.auth().createUser` here — the auth user already exists; use `context.auth.uid` directly.

Safety checks (verified, restate in PR):

- `functions/src/triggers/userCreated.ts` only acts for `userType === "client"` with a phone (line 21) and no-ops if `agent_sessions/{phone}` already exists. The Auth `onCreate` already fired at OTP `confirm()` (before this callable), when no `users` doc existed → it no-ops. This early write does **not** re-trigger `onCreate`, so no duplicate welcome/thread. Caregivers are skipped entirely. Confirm this reasoning still holds after the change.
- `ensureWebAccount` (`onboardingConversation.ts:256`) still runs later at the photo step; its `{ merge: true }` write is harmless on top of the early doc (idempotent). Leave it in place as the cold-SMS path's account creator and as a backstop.

Acceptance:

- Completing web phone-verify for a caregiver creates `users/{uid}` with `userType:"caregiver"`, `phone`, `name` immediately (before any SMS).
- Same for client (`userType:"client"`, `firstName`).
- Re-running the callable does not duplicate/flip fields.

- Add/extend a unit test around `createWebOnboardingSession` asserting the `users/{uid}` write.

B. Stop texting dead placeholder links on API failure — HIGH

Problem: three handlers catch a Checkr/Stripe API error, log it, then **still text the caregiver a fake** `${APP_URL}/done?task=...` link. The caregiver taps a dead link and stalls with no alert and no retry. This is the top “signup silently breaks” risk.

Sites in `functions/src/agents/onboardingConversation.ts` :

1. `handleCaregiverSendBgcheck` — lines 2423-2500 (placeholder set at 2424, sent at 2492).
2. `handleCaregiverSendStripeConnect` — lines 2556-2594 (placeholder at 2558, sent at 2593).
3. `sendBgCheckRenewalLink` — lines 2505-2554 (placeholder at 2506, sent at 2552).

Required behavior for each:

- Track whether a **real** external URL was obtained (e.g. `let realUrl: string | null = null;` set only inside the success branch from the API response).
- If `realUrl` is present → send the link bubble exactly as today.
- If `realUrl` is null (API threw or returned no url) → **do NOT send any link bubble.**

Instead:

- Send an honest, warm Evia line, e.g. via `generateCaraMessage(...)` with a fallback like *“I hit a snag pulling up your background-check link — I’m on it and I’ll text you the moment it’s ready.”* (adapt wording per step: bg-check / payout setup / renewal).
- Write an `admin_alerts` entry, `severity: "high"`, `type: "onboarding_link_generation_failed"`, include `phone`, `step`, and the error message.
- Leave `onboardingStep` at the current awaiting state so the existing `resendStuckStep` stale-session nudge re-enters the handler (it already regenerates real links). Do not advance.

- Remove the `${APP_URL}/done?task=...` fallback as a *sent* value. (You may keep a `/done` deep link only if a real token-backed session was actually created — but for these three the placeholder is never backed by a real session, so it must not be sent.)
- Keep the dry-run `guardSideEffect` stub returns intact (they return a synthetic `invitation_url/link` so replay tests still see a “real” url — that’s correct; the no-send branch only triggers on genuine failure).

Also fold in — Stripe-webhook-driven renewal (different site):

`functions/src/stripe.ts` ~lines 660-737. On the subscription-renewal path, if the Checkr renewal invitation POST fails it currently early-returns with only `console.error` and sends the caregiver **nothing**. Apply the same rule: on failure, send an honest “your annual background check needs a quick renewal — I’ll text your link shortly” message (or queue a retry) **and** write an `admin_alerts` entry. The caregiver must never be left silent on a renewal failure.

Acceptance:

- Simulate Checkr/Stripe throwing in each of the 3 onboarding handlers → caregiver receives an honest message, **no** link bubble, an `admin_alerts` row exists, step unchanged.
 - Success path unchanged (real link still sent, step advances as before).
 - `stripe.ts` renewal failure path sends a message + alert instead of silence.
-

C. Unblock non-clear background checks + fix swallowed clear notice — MED

Problem 1 (dead stop): `advanceOnboardingStep(..., "background_check", ...)` is only called from the Checkr webhook’s `status === "clear"` branch (`functions/src/checkr.ts`, ~lines 518-627). A `consider` / `suspended` / non-clear result parks the caregiver at `caregiver_awaiting_bgcheck` **indefinitely** — there is only an `admin_alerts` entry and no caregiver-facing message or resume path.

Problem 2 (swallowed notice): the renewal-clear notification path (`checkr.ts` ~line 553, inside a try/catch) can silently drop the “your background check just cleared” message on a throw.

Required behavior:

- In the Checkr webhook, explicitly handle non-clear terminal results (`consider` , `suspended` , `dispute` , and any non-`clear` completed status):
 - Send the caregiver a real, warm status message (via `generateCaraMessage` / fallback), e.g. *“Your background check came back needing a closer look. Our team is reviewing it now — I’ll text you as soon as there’s an update. Nothing you need to do right now.”* Do **not** expose adjudication details (Checkr/FCRA sensitivity).
 - Ensure an `admin_alerts` entry exists (`type: "background_check_review"` / `"background_check_suspended"` , `severity: "high"`) with the candidate/report id so an admin can act.
 - Keep bookings paused (do not set `status: "active"` / `verificationStatus: "approved"`).
 - Do not throw from the webhook (return 200 so Checkr doesn’t retry-storm); guard sends with try/catch that still logs.
- For Problem 2: ensure a clear result always attempts the caregiver notification, and on send failure logs + optionally re-queues (the `sendMessage` dead-letter already covers transport failures; the concern is the surrounding try/catch swallowing before the send). Confirm the “you’re cleared” message is actually attempted on both first-time and renewal clears.

Note: there is intentionally no automated “auto-approve on non-clear” — admin stays in the loop. This fix is purely about (a) telling the caregiver something and (b) guaranteeing the alert. Do not build an auto-adjudication path.

Acceptance:

- Simulate a `consider` / `suspended` Checkr webhook → caregiver gets a status message, `admin_alerts` row exists, no `active` / `approved` state written.
- Simulate a `clear` webhook where the notification send would throw → error is logged, state still advances, no unhandled rejection.

D. Text a webapp login link so caregivers reach their dashboard — MED

Problem: signup never texts an “log in at eviacares.com” link. Login is Firebase native phone-OTP the caregiver must self-discover. The only app-domain URL they see is the `/p/{id}` public profile at the very end.

Required behavior:

- The account now exists at phone entry (fix A), so the caregiver can log in immediately.
- Add a login link to the **final** onboarding message in `functions/src/agents/permissionsConversation.ts` (the `caregiver_permissions_arrival` handler, ~lines 314-323, where the `/p/{id}` profile link is already sent). Add a line like: *“Log in anytime to manage your profile, availability, and payouts: {loginUrl}”* where `loginUrl = appLink("/login")` (import from `functions/src/config/appUrl.ts`).
- **Verify the route exists:** confirm `/login` (or the correct auth route) is defined in `App.tsx` and served by `components/auth/LoginPage.tsx`. If the canonical route differs (e.g. `/` with a login modal, or `/caregiver/chat`), use that instead. Do not invent a route.
- Keep it to one extra line; don’t spam a separate bubble unless the profile link and login link read better as two link bubbles (they will auto-split — that’s fine).

Optional (confirm with founder before adding): also surface the login URL in the `createWebOnboardingSession` return payload and/or a very early SMS, since the account exists at phone entry. Default: keep it to the end-of-onboarding message to avoid confusing a mid-flow caregiver.

Acceptance:

- Finishing onboarding sends both the `/p/{id}` profile link and a working login link.
- The login URL resolves to a real route that lets a phone-OTP’d caregiver reach their dashboard.

E. Short bio silently wiped — LOW (cheap)

Problem: `caregiver_ask_bio` sets `bio = ""` when the text is “skip” or under 10 chars (`onboardingConversation.ts:2103`). A genuinely short bio (“Kind and patient.”) is silently discarded with no notice or re-ask.

Required behavior:

- Distinguish an explicit skip (LLM/ `parseWithClaude` intent, per CLAUDE.md — not a raw `.includes("skip")`) from a short-but-real bio.
- If the caregiver typed real content, keep it (drop or lower the 10-char floor; a short bio is still a bio). If it’s genuinely too thin and you want a minimum, re-ask once with a friendly nudge rather than silently wiping.
- **Live-path note:** verify whether bio collection runs through the scripted `handleCaregiverAskBio` or the agent loop (`caregiverFieldAbsorber.ts`). Patch whichever handles it live; if unsure, patch both.

Acceptance: a 3-word bio persists to `onboardingData.bio` / `caregivers.bio`; explicit skip still yields empty with an acknowledgment.

F. MVR mis-config leaves caregiver uninformed — LOW (cheap)

Problem: `wantsMvr` is captured from YES/NO before `canChargeBundledMvr()` is checked. If MVR is unconfigured after the caregiver said yes, the code proceeds membership-only and writes an `admin_alerts` `mvr_signup_misconfigured` — but the caregiver is never told they aren’t getting the MVR add-on they asked for.

Required behavior:

- When `wantsMvr === true` but `canChargeBundledMvr()` is false, send the caregiver one honest line (e.g. *“Heads up — I couldn’t add the driving-record check to your membership right now, so you’re set up with the standard background check. Our team will follow up if you’d like to add it.”*) alongside the existing admin alert.
- **Live-path note:** same as E — the MVR ask may be agent-loop-handled; patch the live handler.

Acceptance: MVR-misconfig path sends a caregiver-facing notice + keeps the admin alert.

Deferred (documented, NOT in this plan — founder decision required to include)

- **Stripe Connect completion has no webhook backfill.** If `account.updated` is lost/delayed, `caregivers/{uid}` never finalizes (only the 7-day `resendStuckStep` nudge regenerates the link; it does not re-check `charges_enabled` / `payouts_enabled`). A polling/backfill or a `resendStuckStep` state re-check is the real fix. Medium lift. `functions/src/stripeConnectWebhook.ts:51-89`.
 - **Early `caregivers/{uid}` creation at phone entry** — founder chose keep-as-is (entangled with Checkr candidate id + Connect finalization + `FindCaregivers` visibility + `firestore.rules`). Larger change.
 - **Cold-SMS account-at-first-text** — founder chose web-only.
 - **CHECKR_API_KEY vs CHECKR_KEY inconsistency** — the 3 direct `axios.post` calls in `onboardingConversation.ts` (2445, 2530, ~2700) read `CHECKR_API_KEY` only, while `checkr.ts` / `stripe.ts` prefer `CHECKR_KEY`. Harmless today (`.env` populates `CHECKR_API_KEY`, `CHECKR_KEY` empty); latent break if the key source migrates. One-line each if you want to harden.
 - **`caregiver_pending_review` alert naming misleading** (`permissionsConversation.ts:330-338`) — fires after the caregiver is already `active` / `approved` from bg-check clear. Cosmetic/labeling only.
 - **`firstName` vs `name` split fragility** — documented invariant; any new caregiver text-correction code must remap `firstName→name` (see `detectCorrection`, `onboardingConversation.ts:697-707`).
 - **`stripe_connect` finalization branch** (`onboardingConversation.ts:3212-3441`) is the most complex in the file (3 caregiverId-resolution paths + legacy-doc migration). Thin characterization test coverage — worth adding tests, not behavior change.
-

Suggested implementation order

1. **A** (account at phone entry) — isolated, in `index.ts`; add test.

2. **B** (dead links + `stripe.ts` renewal) — highest user impact.
3. **C** (non-clear BG + clear-notice) — in `checkr.ts`.
4. **D** (login link) — verify route first.
5. **E, F** (cheap collection-step fixes; check live agent-loop handler).
6. Build + tests + report. Do not deploy.

Definition of done

- `npm --prefix functions run build` passes.
- Function unit tests pass (excluding the 3 pre-existing `onboardingReplay` failures).
- New/updated tests cover A (users doc write), B (no-link-on-failure + alert), C (non-clear message + alert).
- No new user-facing string parses intent with regex/keywords (CLAUDE.md rule).
- PR description lists exactly which handlers were confirmed on the live agent-loop vs scripted path for E/F.